

第三章 系统分析

3.1 可行性分析

项目启动前依惯例完成三方面论证^[9]。技术可行性：React 与 Ink 文档完备、社区活跃，Bun 一键安装运行，JavaScript 语言门槛低；数据层仅需文件读写接口，无外部依赖。经济可行性：全部工具与运行时免费，无服务器与数据库开销，任何个人计算机均可运行。操作可行性：终端界面以键盘操作为主，配合屏幕提示与快捷键说明，熟悉命令行的用户可即时上手；系统同时内置演示账号与种子数据，便于快速体验。三方面均无障碍，项目可行。

3.2 需求概述与事件列表

系统角色分为普通用户与管理员。普通用户围绕内容消费与创作：注册登录后浏览首页信息流，可按频道（类型）切换、按关键字搜索、切换排序方式并翻页；进入详情页后可点赞（再次操作取消）、发表评论或回复他人评论；也可发布头条、修改或删除自己发布的内容，维护个人密码，查看公告。管理员在普通能力之外，通过管理面板对头条、类型、公告、评论进行全量治理，并查看热门与活跃排行。将上述业务整理为系统必须响应的事件，得到表 2.1 的事件列表。

表 3.1 系统事件列表

事件名称	触发条件	系统响应（对应用例）	参与者
注册账号	提交用户名、密码与昵称	三重校验后建户（角色为普通用户）	访客
登录	提交用户名与密码	匹配账户并进入主界面	普通用户 / 管理员
修改密码	提交旧密码与两遍新密码	旧密码校验后更新	普通用户
浏览信息流	进入首页或切换条件	按类型 / 关键字过滤，排序分页展示	普通用户
查看详情	选中头条打开	浏览计数加一并展示评论	普通用户
点赞 / 取消	在详情页触发点赞	记录防重，计数同步增减	普通用户
发表评论 / 回复	提交评论内容（可指定父评论）	校验长度后保存，评论计数加一	普通用户
发布头条	填写标题、内容并选择类型	校验后保存，计数清零入库	普通用户
维护我的头条	修改或删除自己的头条	归属校验，删除级联清理	普通用户
查看公告	进入公告页	按时间倒序列示，支持检索	普通用户
头条治理	管理员改删任意头条	权限放行，删除级联清理	管理员
类型 / 公告管理	管理员维护类型与公告	增改删；类型有关联禁删	管理员
排行与评论管理	管理员查看排行 / 删评论	多口径排序；评论级联删除	管理员

3.3 业务流程分析（活动图）

普通用户的主流程如图 3.1 所示：登录成功后进入首页信息流，在频道切换、搜索与排序中定位感兴趣的内容；打开头条详情会使浏览计数加一，随后可点赞或发表、回复评论；判断是否继续浏览，是则回到信息流，否则可发布或维护自己的头条、查看个人中心与公告，最后退出系统，数据随每次写操作即时落盘。登录失败则停留在登录页重新输入。

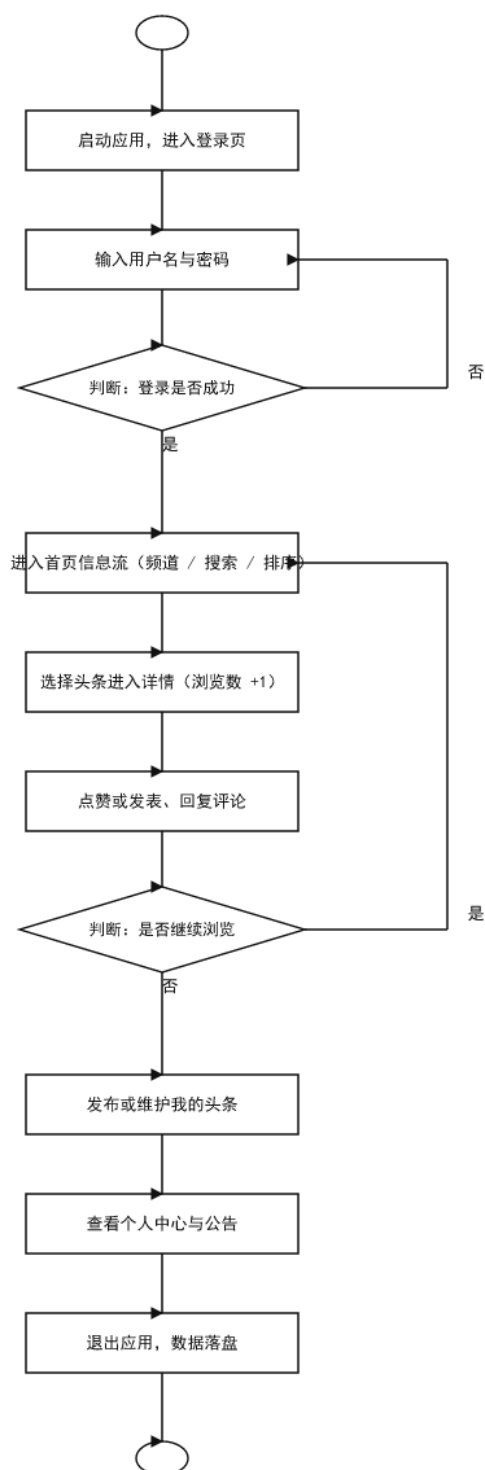


图 3.1 普通用户浏览互动活动图

管理员的治理流程如图 3.2 所示：管理员登录后进入管理面板，在头条全量管理、类型与公告管理、排行与评论管理三类任务间切换；其中头条与评论的删除均执行级联清理，类型的删除受关联头条保护；全部操作完成后退出系统。

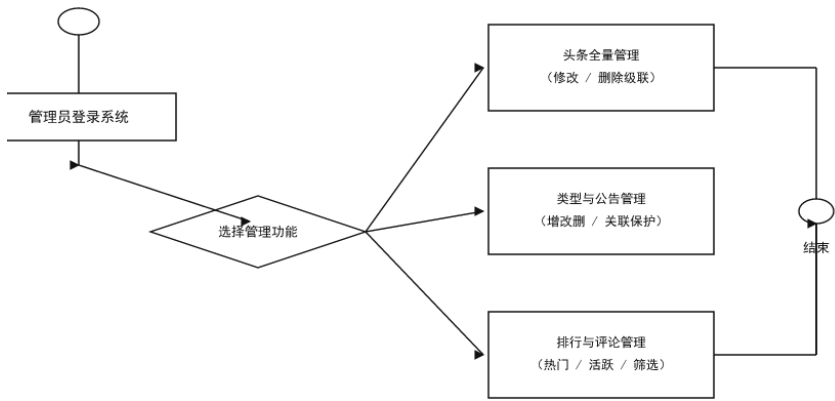


图 3.2 管理员治理活动图

3.4 用例建模：用例图与用例描述

将事件列表归并提炼，得到系统用例图，如图 3.3 所示。普通用户侧包含账户（注册、登录、改密）、浏览（信息流、搜索筛选、详情、公告）、互动（点赞、评论、回复）与创作（发布、修改、删除我的头条）四组用例；管理员侧为头条、类型、公告、排行、评论五类治理用例。

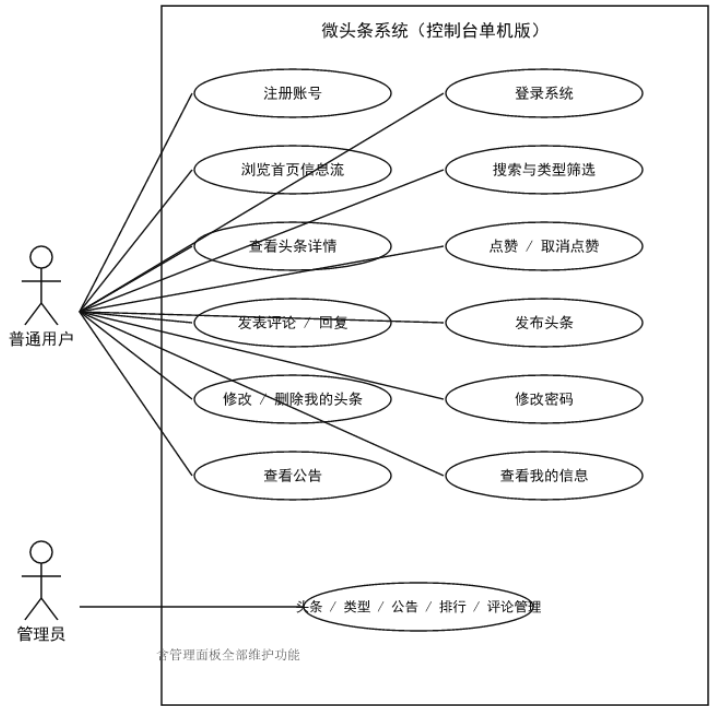


图 3.3 系统用例图

选取发布头条、点赞互动与类型管理三个代表性用例编写描述表，见表 3.2 至表 3.4，分别覆盖创作类、互动类与治理类业务。

表 3.2 用例描述：发布头条

用例项目	内容
用例名称	发布头条
参与者	普通用户
前置条件	已登录；系统中存在可选类型
主流程	1. 在发布页逐项填写标题与内容；2. 左右键切换类型；3. 确认发布；4. 系统校验非空、长度与类型有效性；5. 保存入库（三项计数清零）；6. 返回并可在信息流中看到新内容
备用流程	任一校验不通过时提示原因，停留发布页
后置条件	新头条持久化保存，作者昵称随账户解析

表 3.3 用例描述：点赞互动

用例项目	内容
用例名称	点赞互动
参与者	普通用户
前置条件	已登录并进入头条详情
主流程	1. 触发点赞；2. 系统查找该用户对该头条的点赞记录；3. 无记录则新增并使点赞数加一；4. 有记录则移除并使点赞数减一（不低于零）；5. 结果落盘并即时刷新界面
备用流程	头条不存在时返回提示
后置条件	点赞记录与点赞计数严格一致

表 3.4 用例描述：类型管理

用例项目	内容
用例名称	类型管理
参与者	管理员
前置条件	以管理员身份登录
主流程	1. 进入类型管理；2. 新增（查重）或重命名类型；3. 保存并刷新频道列表
备用流程	删除类型时若其下存在头条，拒绝并提示；类型名为空或重复时提示
后置条件	类型数据更新，频道入口即时反映

3.5 静态结构分析：仅含属性的静态类图

从用例描述中提取业务实体，得到用户、类型、头条、点赞记录、评论、公告六个实体，其属性构成图 3.4 所示的静态类图。关系方面：用户与头条、用户与点赞、用户与评论均为一对多；类型与头条为一对多的归类关系；评论以父评论标识支持两级回复结构；头条冗余保存浏览、点赞、评论三项计数以支撑高频展示。

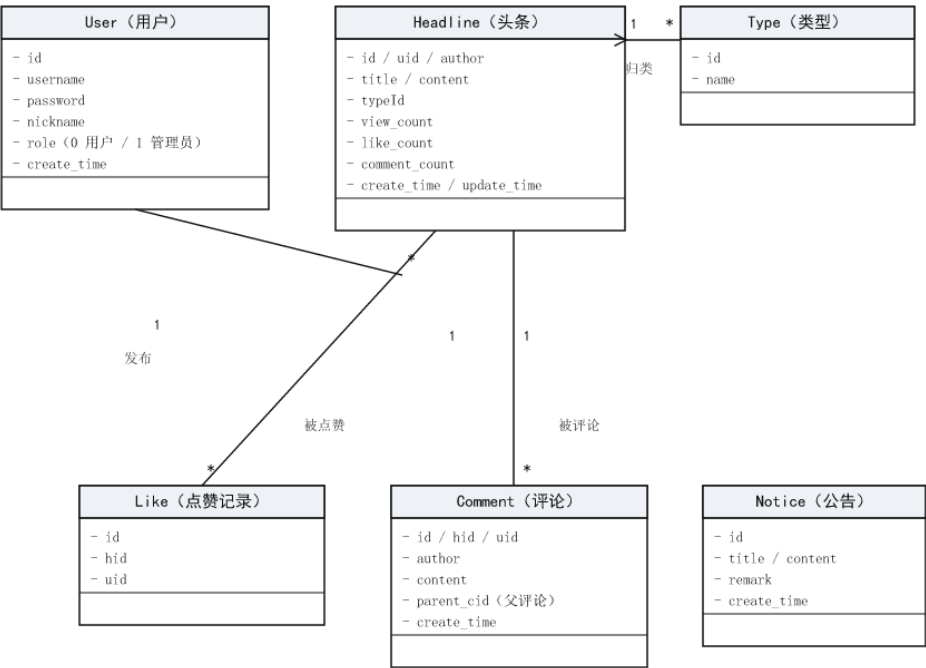


图 3.4 实体静态类图（仅属性）

3.6 动态行为分析：顺序图与完整类图

为明确职责流转，对登录与点赞两个核心场景建立顺序图。图 3.5 展示了点赞场景：详情组件响应用户按键，业务对象先在点赞集合中查找（用户、头条）二元组记录，据此决定新增或移除并同步冗余计数，再经临时文件加重命名落盘，最后把结果回传界面即时刷新；重复触发即切换为取消，天然实现防重。图 3.6 展示了登录场景：登录组件提交凭据，业务对象在用户集合中匹配，成功则更新全局状态并切换渲染主界面，失败则提示并停留。

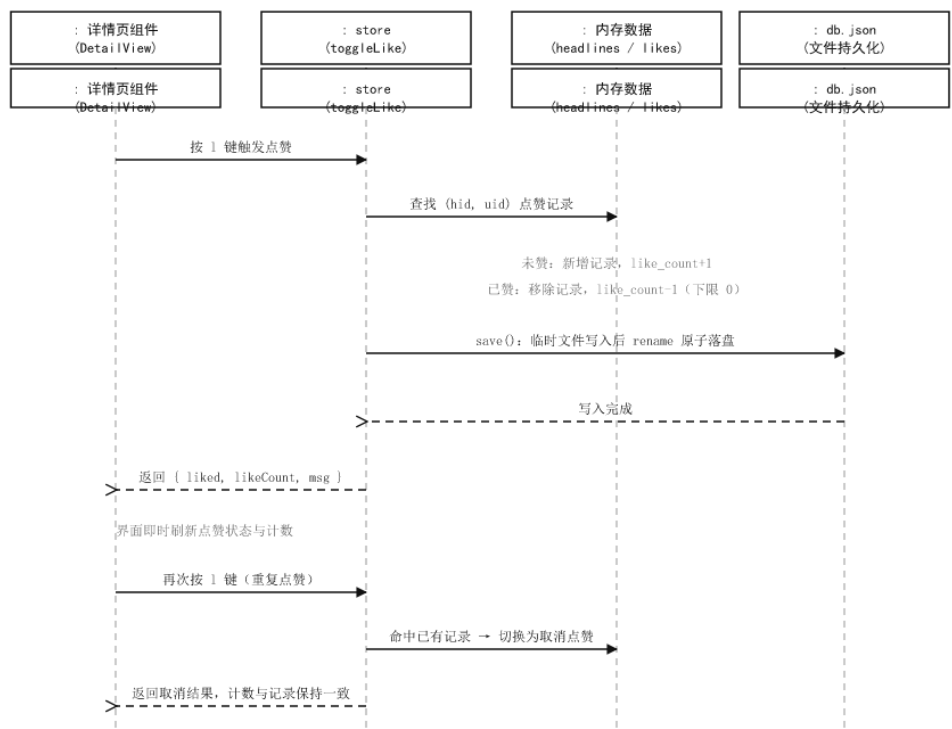


图 3.5 点赞互动顺序图

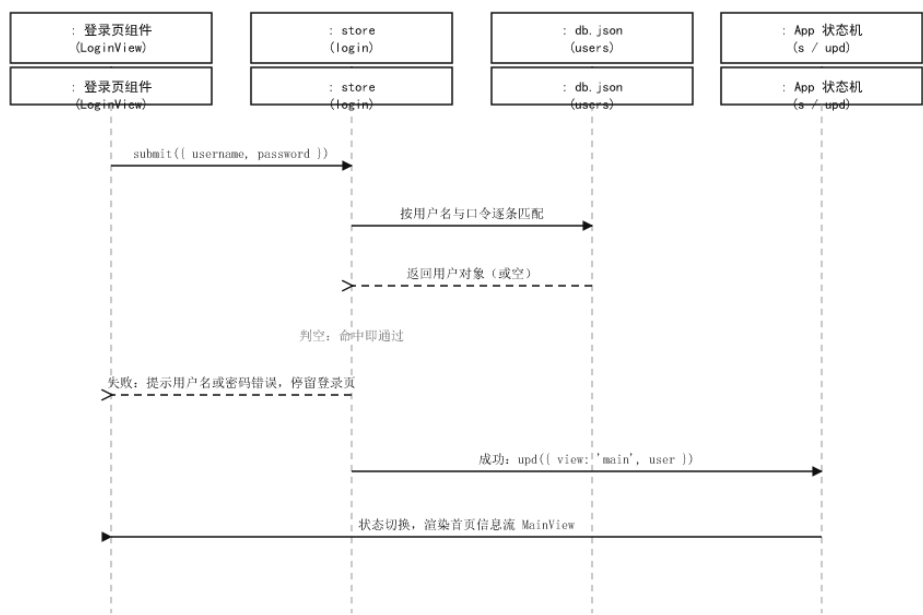


图 3.6 用户登录顺序图

将顺序图中的职责固化到结构上，得到图 3.7 的完整设计类图：视图组件层以 App 状态机为枢纽，各视图组件只负责呈现与按键采集；store 业务对象承接全部业务规则，按认证、查询、维护、互动、管理五组方法组织；持久化函数负责加载、种子与原子保存。至此分析与设计的衔接完成。

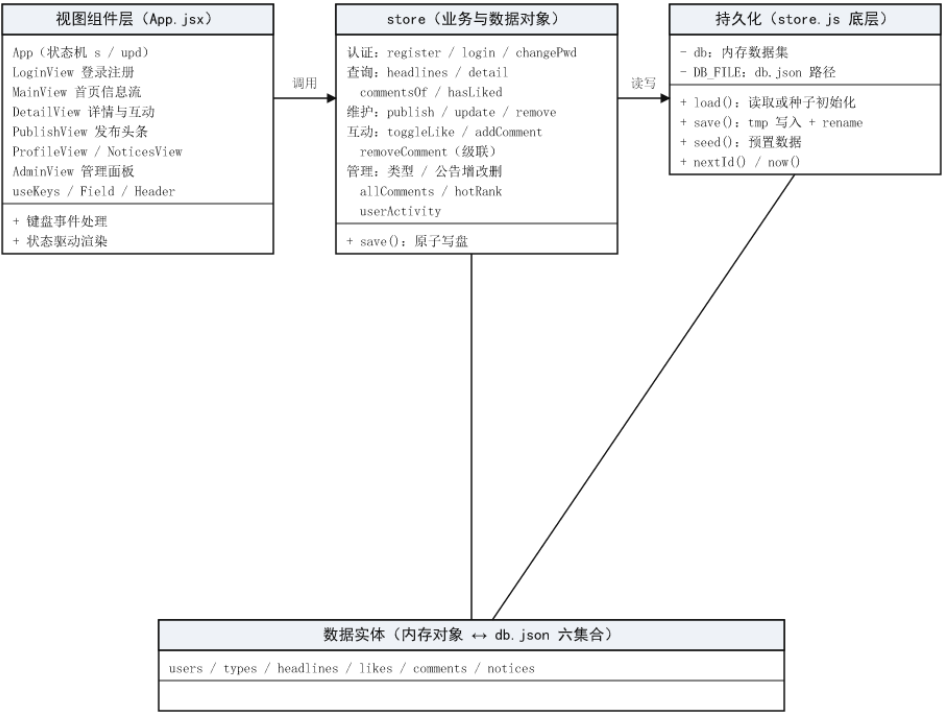


图 3.7 设计类图（属性与方法）

第四章 系统设计

4.1 体系结构设计

4.1.1 逻辑结构设计

系统逻辑上分为三层，如图 4.1 所示。视图组件层运行于 React 18 与 Ink 5 之上，包含登录、首页信息流、详情、发布、个人中心、公告与管理面板七类组件，以集中状态机驱动切换^[11]。业务数据层即 store 模块，封装认证账户、头条查询维护、点赞评论、类型公告与排行等全部业务规则，是视图与数据之间唯一的通道。持久化层将六个数据集合以 JSON 单文件形式保存^[10]，加载时解析或初始化种子，保存时以临时文件加重命名方式整体写出，保障落盘原子性。三层之间自上而下单向调用，任何一层的替换均不影响其余两层。

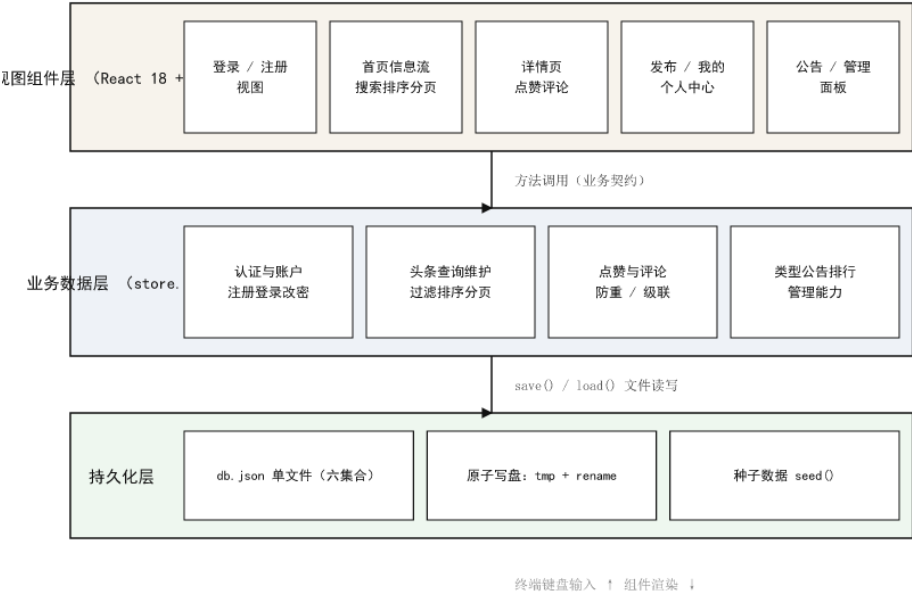


图 4.1 系统逻辑结构示意图

4.1.2 物理结构设计

系统为纯单机应用，物理形态如图 4.2 所示：用户计算机上安装 Bun 1.3 及以上运行时，微头条 TUI 应用（React 18 加 Ink 5 组件与 store 模块）以解释方式运行，数据文件 db.json 与应用同目录存放；终端设备承担字符界面的渲染与键盘输入。系统不依赖任何网络服务，删除数据文件即可恢复种子初始状态。

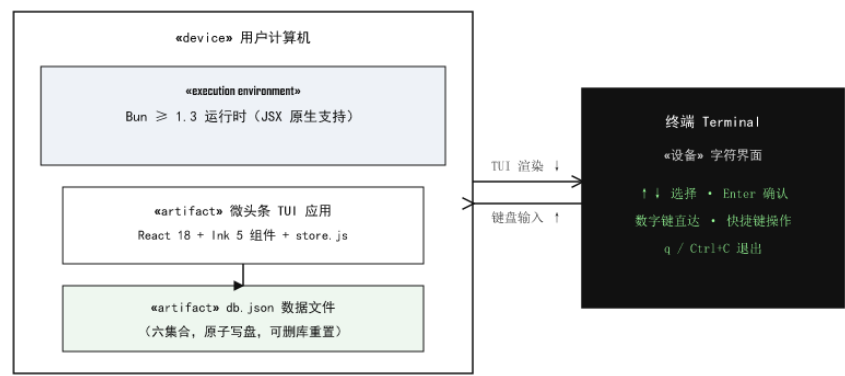


图 4.2 系统物理部署图

4.2 功能模块设计

按职责单一原则，系统功能划分为六个模块，如图 4.3 所示：认证模块、资讯浏览模块、互动模块、内容发布模块、个人与公告模块、管理模块。各模块业务规则全部沉淀在 store 对应方法组中，视图组件按模块组织，模块间不产生横向依赖。

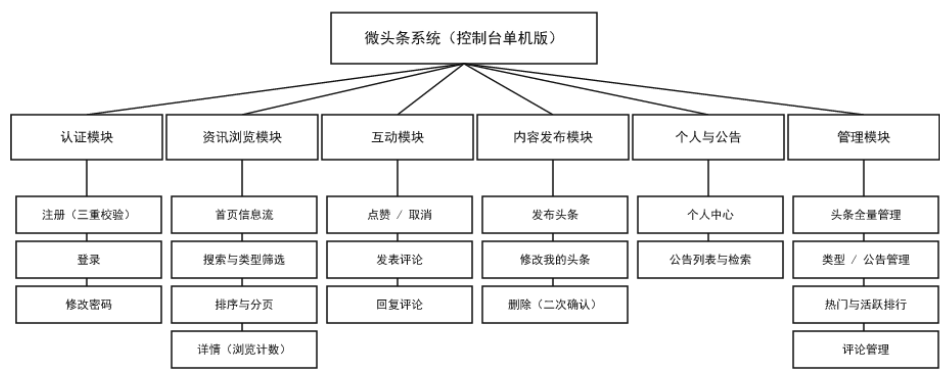


图 4.3 系统功能模块结构图

4.3 类与模块设计

视图与数据的职责分配见表 4.1。设计要点有三：其一，App 以单一状态对象与更新函数构成状态机，视图切换、弹层与消息均纳入状态，避免组件间零散传参；其二，store 是唯一的业务入口，视图组件不直接接触数据文件；其三，权限校验（本人或管理员）内聚在维护类方法中，由方法签名携带角色参数统一裁决。

表 4.1 模块与职责设计

构件	层次	职责与关键成员
App（状态机）	视图层	全局状态 s 与更新函数 upd，驱动视图切换
LoginView / MainView	视图层	登录注册表单；信息流渲染与条件交互
DetailView / PublishView	视图层	详情与评论互动；头条发布表单
ProfileView / NoticesView	视图层	个人中心改密；公告列表
AdminView	视图层	管理面板：头条 / 类型 / 公告 / 排行 / 评论
useKeys / Field / Header	视图层	键盘事件注册；输入域；页头与提示
store 认证方法组	业务层	register / login / changePwd
store 查询方法组	业务层	headlines（过滤排序分页）/ detail / commentsOf / hasLiked
store 维护与互动组	业务层	publish / update / remove / toggleLike / addComment / removeComment
store 管理方法组	业务层	类型与公告增改删 / allComments / hotRank / userActivity
load / save / seed	持久化层	读取或种子初始化；原子写盘；预置数据

4.4 数据库设计

4.4.1 数据库概要设计

概念结构设计是将需求分析中识别出的业务实体及其相互关系，抽象为独立于具体数据库管理系统的概念模型。在对本系统进行数据库概念设计时，充分考虑到了系统的可扩展性和灵活性，以便在未来对系统进行修改和扩展时能够更加方便。同时，遵循了数据库设计的基本原则。本系统采用实体-联系图（E-R 图）作为概念模型的表达工具。以下逐一描述各核心实体的属性构成，最后给出系统整体的 E-R 图。

(1)用户实体

用户实体用于存储系统注册用户的基本账号信息与权限数据。该实体的属性包括：用户 ID（全局唯一主键）、用户名（登录账号）、密码（经加密处理的密文）、昵称（显示名称）、角色（区分学生、管理员等身份）、创建时间（账号注册时间）。用户实体是系统权限控制的基础，通过角色字段实现不同身份的功能隔离。用户实体属性图如图 4.4 所示。

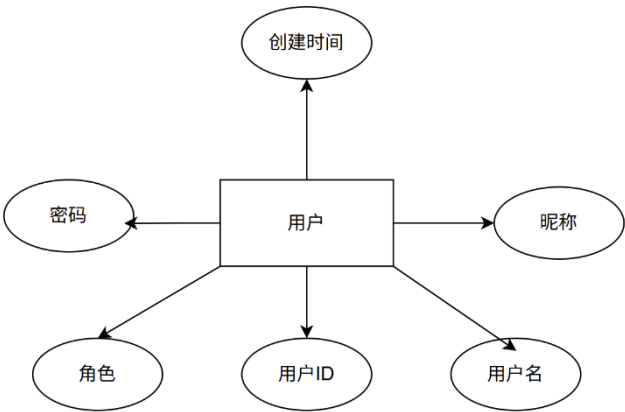


图 4.4 用户实体属性图

(2)评论实体

评论实体用于存储用户对系统内容（如头条）的评论及回复信息，支持多层级的互动交流。该实体的属性包括：评论 ID（唯一标识主键）、用户 ID（关联评论者）、头条 ID（关联被评论内容）、评论者（评论人昵称）、评论内容（具体文字）、父评论 ID（关联上级评论，用于实现回复功能）、创建时间（评论发布时间）。该实体通过用户 ID 和头条 ID 关联核心业务表，通过父评论 ID 实现评论的嵌套与追溯。评论实体属性图如图 4.5 所示。

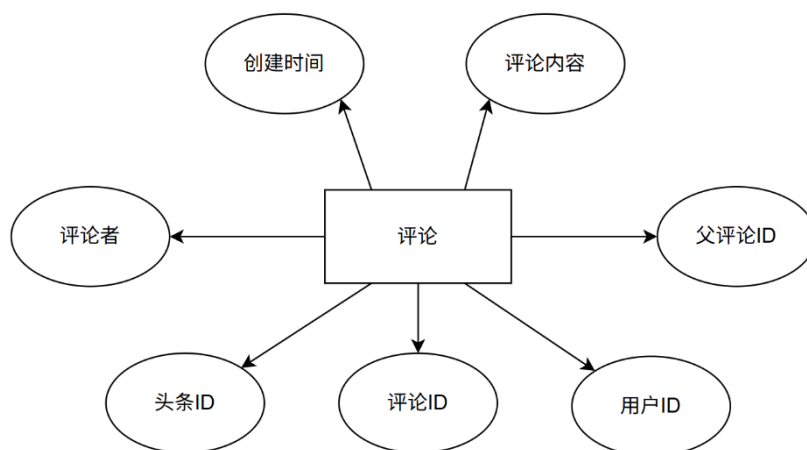


图 4.5 评论实体属性图

(3)头条实体

头条实体用于存储系统发布的资讯、文章或公告类核心内容。该实体的属性包括：头条 ID（唯一标识）、标题（内容概要）、内容（正文详情）、类型 ID（关联分类信息）、发布者 ID（关联发布用户）、作者（署名）、浏览量（阅读次数统计）、点赞数（互动数据）、评论数（互动数据）、创建时间（首次发布时间）、更新时间（最后修改时间）。该实体是内容管理模块的核心，通过外键关联用户和类型实体，支撑内容的发布、展示与统计功能。学生用户实体属性图如图 4.6 所示。

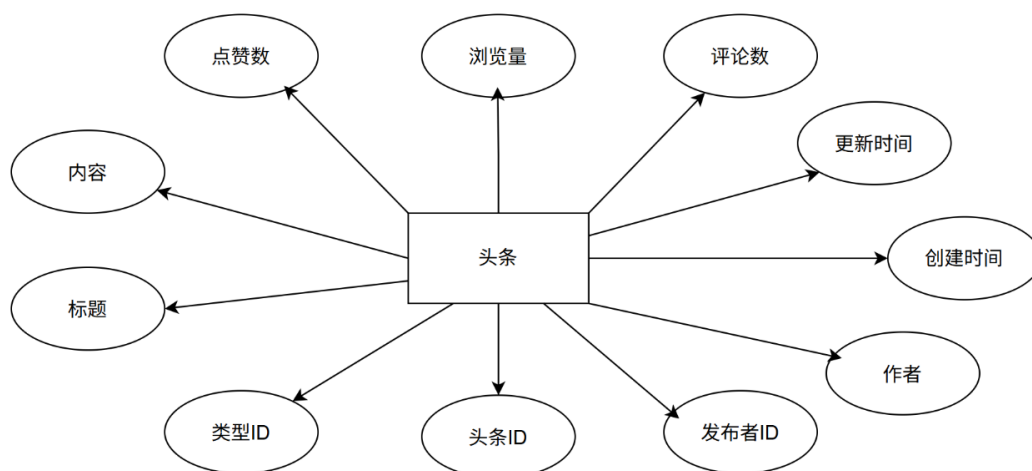


图 4.6 头条实体属性图

(4)类型实体

类型实体用于对系统中的内容（如头条）进行归类管理，便于用户按类别浏览和检索。该实体的属性包括：类型 ID（唯一标识主键）、类型名称（分类标签，如“校园新闻”“自习室通知”等）。类型实体与头条实体之间为一对多的联系，一个类型下可以包含多条头条内容。类型实体属性图如图 4.7 所示。

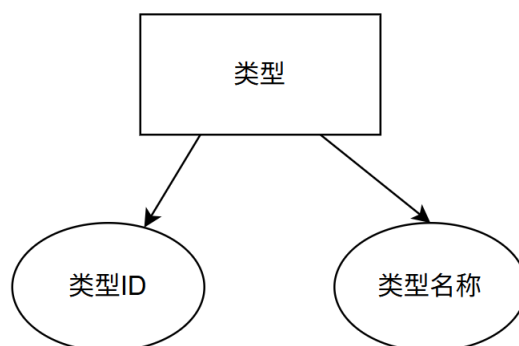


图 4.7 类型实体属性图

(5)点赞实体

点赞记录实体用于存储用户对特定内容（如头条）的点赞行为，作为互动统计和用户偏好分析的数据基础。该实体的属性包括：记录 ID（唯一标识主键）、用户 ID（关联点赞用户）、头条 ID（关联被点赞的内容）。该实体通过用户 ID 和头条 ID 分别关联用户表和头条表，共同构成多对多的互动关系映射，确保同一用户对同一内容不可重复点赞。点赞记录实体属性图如图 4.8 所示。

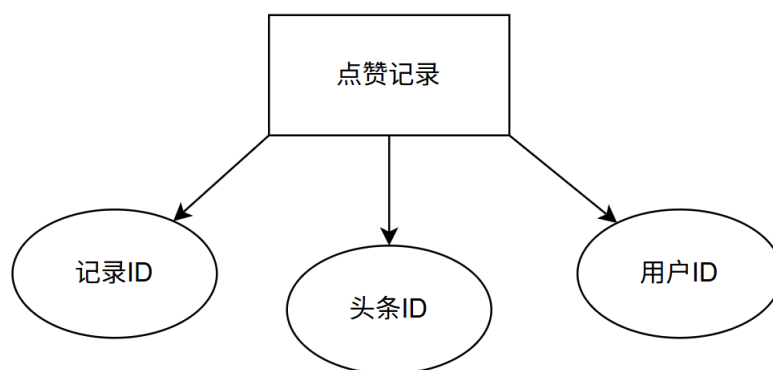


图 4.8 点赞记录实体属性图

(6) 公告实体

公告实体用于存储管理员发布的系统公告信息。该实体的属性包括：公告 ID（唯一标识主键）、标题（公告名称）、内容（公告正文）、备注（附加说明或补充信息）、创建时间（发布时间）。公告信息将展示在系统前端首页或公告栏区域，用于通知用户有关系统调整、临时闭馆、规则变更等重要信息。公告实体属性图如图 4.9 所示。

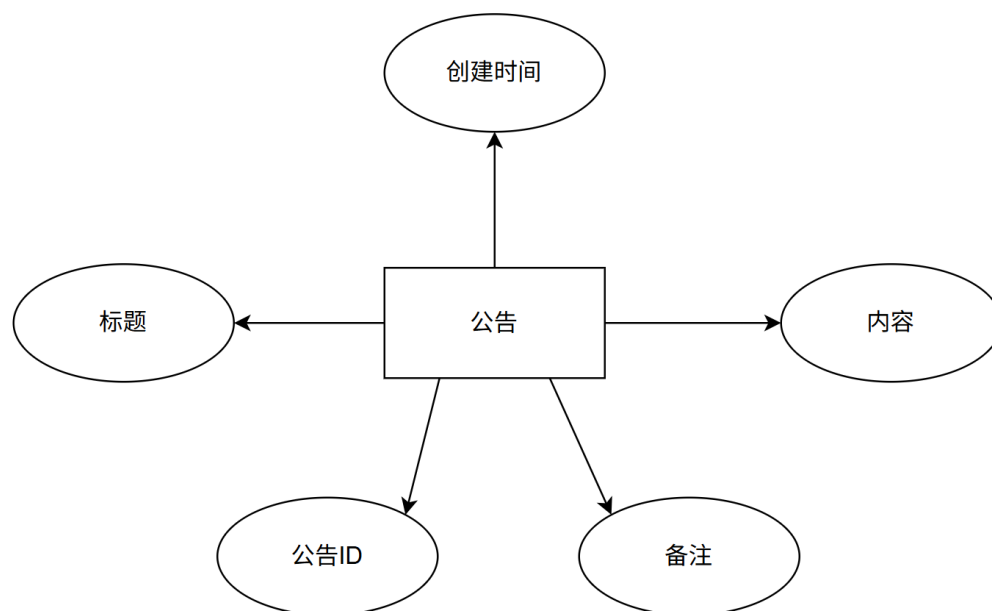


图 4.9 公告实体属性图

(7) 系统整体 E-R 图

综合上述各实体的分析，可以绘制系统整体的 E-R 图。在整体 E-R 图中，各实体之间的关系如下：用户实体与头条实体之间为一对多的联系，一个用户可以发布多条头条内容；用户实体与评论实体之间为一对多的联系，一个用户可以发表多条评论；用户实体与点赞记录实体之间为一对多的联系，一个用户可以进行多次点赞操作；类型实体与头条实体之间为一对多的联系，一个类型下可以包含多条头条内容；头条实体与评论实体之间为一对多的联系，一条头条内容可以关联多条评论；头条实体与点赞记录实体之间为一对多的联系，一条头条内容可以被多个用户点赞；评论实体与自身之间存在一对多的自关联联系，一条父评论可以包含多条子评论（回复），用于实现评论的嵌套与追溯；公告实体相对独立，由管理员进行发布与维护，不与其他业务实体直接关联系统整体 E-R 图如图 4.10 所示。

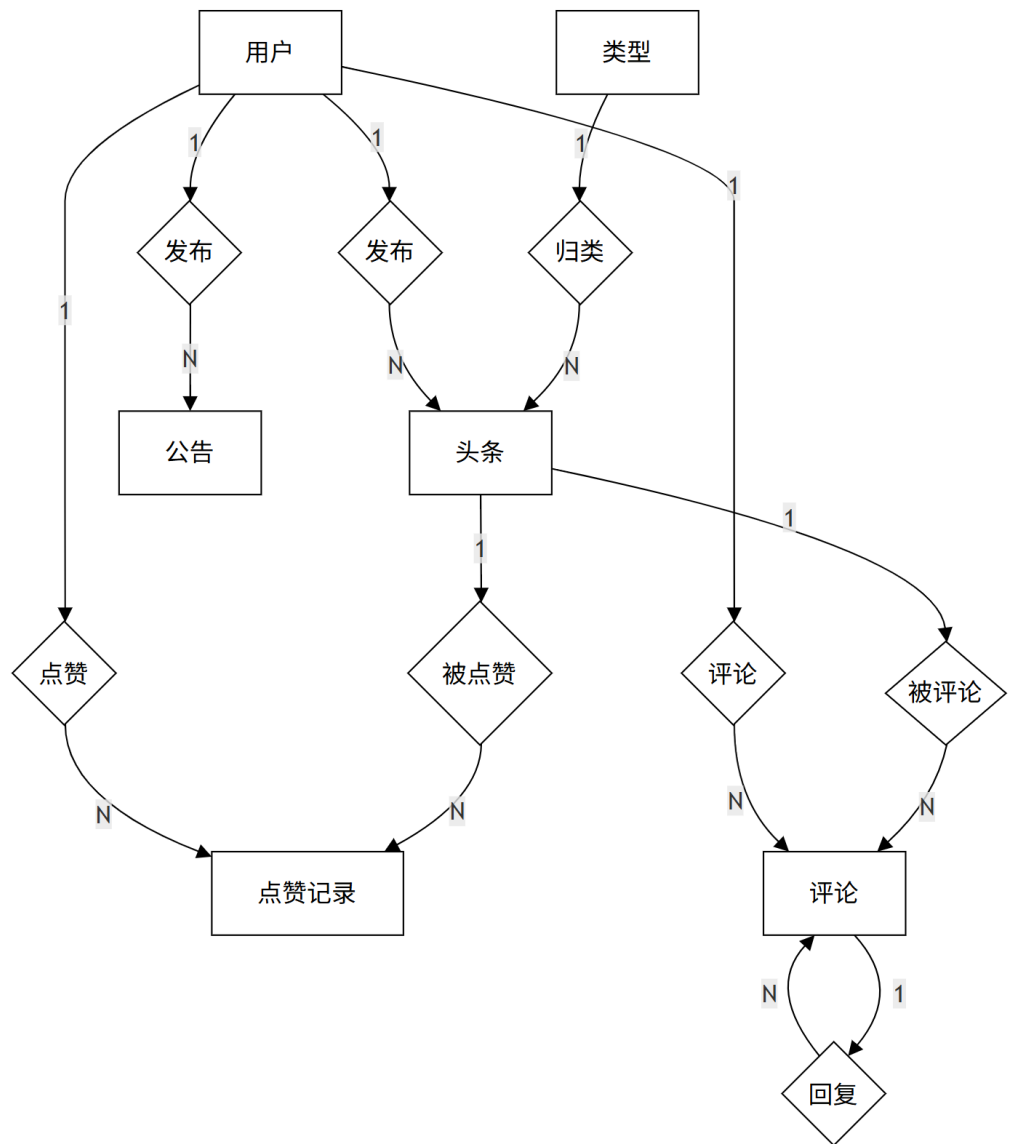


图 4.10 系统整体 E-R 图

4.4.2 数据库逻辑设计

本系统采用本地 db.json 文件完成数据持久化，未引入传统关系型数据库，但为清晰界定领域实体的字段结构、关联约束与业务规则，参照关系数据库范式设计六张逻辑数据表，分别对应用户、头条、类型、点赞记录、评论、公告六大实体。各表通过主键字段唯一标识记录，通过外键建立实体间的关联关系；部分展示类字段采用冗余存储设计，避免查询时频繁联表，适配单机一体化系统的查询性能需求。

(1) 用户表

存储系统全部注册用户基础信息，区分普通用户与管理员角色；用户名字段设置唯一约束，作为登录账号用于身份校验；role 字段用于控制后台管理面板的访问权限。

表 4.2 用户表				
字段名称	类型	长度	主键	备注
id	bigint	20	是	用户 ID
username	varchar	50	否	用户名（登陆账号键，唯一）
password	varchar	255	否	密码
nickname	varchar	50	否	用户昵称
role	varchar	20	否	用户角色（user 普通用户/admin 管理员）
created_at	datetime	-	否	创建时间

（2）头条表

保存用户发布的微头条资讯主体内容；关联类型表实现内容分类，关联用户表标记发布者；冗余存储作者昵称以省去关联查询；维护点赞、浏览、评论三项统计计数字段，用于列表排序与展示；内容修改时自动更新更新时间字段。

表 4.3 头条表				
字段名称	类型	长度	主键	备注
id	bigint	20	是	头条 ID
title	varchar	255	否	标题
content	text	-	否	内容
type_id	bigint	20	否	类型 ID(外键，关联类型表)
author_id	bigint	20	否	发布者 ID（外键，关联用户表）
author_name	varchar	50	否	作者昵称（冗余存储，避免联表查询）
like_count	int	11	否	点赞数
view_count	int	11	否	浏览量
comment_count	int	11	否	评论数
created_at	datetime	-	否	创建时间
updated_at	datetime	-	否	更新时间

（3）类型表

微头条内容分类字典表，由管理员端维护；头条记录通过 `type_id` 字段关联本表；当存在头条数据引用该类型记录时，禁止删除对应类型，防止产生无类型归属的孤岛数据。

表 4.4 类型表

字段名称	类型	长度	主键	备注
id	bigint	20	是	类型 ID
name	varchar	50	否	更新时间

（4）点赞记录表

记录用户对头条的点赞行为；联合 `headline_id` 与 `user_id` 字段形成唯一约束，保证同一个用户对同一条头条只能点赞一次；删除头条时级联清除本表中对应的全部点赞记录，保持数据一致性。

表 4.5 点赞记录表

字段名称	类型	长度	主键	备注
id	bigint	20	是	记录 ID
headline_id	bigint	20	否	头条 ID（外键，关联头条表）
user_id	bigint	20	否	用户 ID（外键，关联用户表）

（5）评论表

存储头条下的评论及评论回复数据；通过 `parent_id` 字段实现两级评论回复结构，取值为 0 代表顶层主评论，非 0 代表回复对应父评论；冗余存储评论者昵称；头条删除时级联删除全部下属评论，删除父评论时同步清理子回复，同时同步更新头条表的评论计数字段。

表 4.6 评论表

字段名称	类型	长度	主键	备注
id	bigint	20	是	评论 ID
content	varchar	1000	否	评论内容
commentator	varchar	50	否	评论者昵称（冗余存储）
user_id	bigint	20	否	用户 ID（外键，评论发布者）
headline_id	bigint	20	否	头条 ID（外键，所属头条）
parent_id	bigint	20	否	父评论 ID，0 表示顶级评论（支持两级回复）
created_at	datetime	-	否	创建时间

（6）公告表

系统公告数据表，仅管理员可进行新增、修改、删除操作；首页公告栏读取本表数据进行展示；remark 字段用于管理员内部备注，不对普通用户前端展示。

表 4.7 公告表

字段名称	类型	长度	主键	备注
id	bigint	20	是	公告 ID
title	varchar	255	否	标题
content	text	-	否	内容
remark	varchar	255	否	备注
created_at	datetime	-	否	创建时间

注：本系统并未使用数据库

4.5 数据与接口契约设计

系统虽无网络接口，但视图层与业务层之间以 store 方法为契约，其规范程度直接决定组件的可维护性。核心方法契约见表 4.8，约定了参数、返回与约束：查询类方法返回数据或带分页信息的结果对象；写操作方法以空值表示成功、以错误文案表示失败，调用方据此呈现提示；所有写操作在返回前完成落盘。

表 4.8 store 核心方法契约

方法	参数	返回 / 结果	约束说明
register	用户名、密码、确认、昵称	空值成功 / 错误文案	非空、两次一致、长度不小于六、用户名唯一
login	用户名、密码	用户对象或空	逐条匹配用户名与口令
headlines	关键字、类型、作者、排序、页码	列表加总数与分页信息	排序支持时间、点赞、评论、浏览四种口径
detail	头条编号	头条对象或空	返回前浏览计数加一并落盘
toggleLike	头条号、用户号	状态、计数与消息	以（用户，头条）二元组防重，计数联动
addComment	头条号、用户号、内容、父评论号	空值成功 / 错误文案	内容非空且不超过五百字，评论计数加一
removeComment	评论号、用户号、角色	空值成功 / 错误文案	限本人或管理员；递归删除子评论并同步计数
publish / update / remove	头条数据与操作者	空值成功 / 错误文案	改删限本人或管理员；删除级联清理点赞与评论
removeType	类型编号	空值成功 / 错误文案	类型下存在头条时禁止删除
hotRank / userActivity	排序口径、页码	排行列表	头条按赞评浏排序；用户按发帖加评论活跃度排序

4.6 键盘交互与界面设计

终端界面没有鼠标参与，键盘交互的设计质量直接决定可用性。本系统确立了三条交互原则：方向键加回车覆盖全部基础导航，任何列表均可上下选择、回车进入；高频操作配备单键快捷键，且随页面语境变化；破坏性操作（删除）一律二次确认。各页面的快捷键设计见表 4.9，界面采用组件化的分区布局：页头呈现当前用户与页面标题，中部为列表或表单区，底部常驻操作提示，输入域以密文样式回显密码。

表 4.9 键盘快捷键设计

页面 / 场景	按键	功能
通用	上下方向键 / Enter / 数字键 / q	选择 / 确认进入 / 菜单直达 / 退出
首页信息流	t / s / 斜杠 / 中括号	循环切频道 / 切排序 / 搜索过滤 / 翻页
首页信息流	n / m / p / o / a	发布 / 我的头条 / 个人中心 / 公告 / 管理面板
头条详情	l / i / r / d	点赞 / 写评论 / 回复选中 / 删除选中评论
我的头条	e / x	修改（类型不可改） / 删除并二次确认
发布页	Tab / 左右方向键	逐项填写 / 切换类型
管理面板	c / x / t / w	改选中 / 删选中 / 切类型 / 发公告

第五章 系统实现

5.1 运行环境与工程结构

开发与运行环境为：Windows 11 操作系统；Bun 1.4 运行时（要求 1.3 以上，原生支持 JSX）；依赖仅 React 18 与 Ink 5 两项，安装即用，体现了单机版极简的工程形态。工程由三个源文件构成：index.js 为入口，调用渲染器挂载应用；App.jsx 实现全部视图组件与状态机；store.js 实现业务规则与持久化。数据文件 db.json 随首次运行自动生成。

5.2 注册与密码修改实现

注册接口按先易后难的顺序执行三重校验，再叠加长度与唯一性约束，任何一步失败都返回面向用户的中文提示。实现节选如下：

```
register({ username, password, confirm, nickname }) {  
  if (!username || !password || !nickname) return '用户名、密码、昵称不能为空'  
  if (password !== confirm) return '两次输入的密码不一致'  
  if (password.length < 6) return '密码长度至少 6 位'  
  if (db.users.some(u => u.username === username)) return '用户名已存在'  
  const user = { id: nextId(db.users), username, password,  
    nickname, role: 0, create_time: now() }  
  db.users.push(user); save()  
  return null  
}
```

密码修改同样先校验旧口令，再校验新口令长度与两遍一致，全部通过后更新并落盘，返回值约定与注册一致，视图层无分支即可复用提示逻辑。

5.3 信息流查询实现

信息流查询是系统使用频率最高的路径，实现上采用先过滤、再排序、后分页的流水线：关键字在标题与正文中模糊匹配，类型与作者按需叠加；排序器以字典提供四种口径，键不存在时回退到时间排序；最后按页容量切片并连同总数返回，供界面渲染分页条。实现节选如下：

```
const sorters = {  
  time: (a, b) => b.create_time.localeCompare(a.create_time),  
  like: (a, b) => b.like_count - a.like_count,  
  comment: (a, b) => b.comment_count - a.comment_count,  
  view: (a, b) => b.view_count - a.view_count,  
}  
list.sort(sorters[sort] || sorters.time)  
const total = list.length  
list = list.slice((page - 1) * pageSize, page * pageSize)  
return { list, total, page, pageSize }
```

5.4 点赞防重与计数同步实现

点赞的难点在于防重与计数一致。实现以点赞集合中的（头条，用户）二元组为唯一依据：

命中即视为已赞，本次操作转为取消；未命中则新增点赞并使计数加一。计数随记录在同一内存对象上同步修改，并在返回前统一落盘，相当于以单线程写操作模拟了事务语义。实现节选如下：

```
toggleLike(hid, uid) {
  const h = db.headlines.find(h => h.id === hid)
  if (!h) return { liked: false, likeCount: 0, msg: '头条不存在' }
  const i = db.likes.findIndex(l => l.hid === hid && l.uid === uid)
  if (i >= 0) {
    db.likes.splice(i, 1)
    h.like_count = Math.max(0, h.like_count - 1)
    save(); return { liked: false, likeCount: h.like_count, msg: '已取消点赞' }
  }
  db.likes.push({ id: nextId(db.likes), hid, uid })
  h.like_count++; save()
  return { liked: true, likeCount: h.like_count, msg: '点赞成功' }
}
```

5.5 级联删除与原子写盘实现

评论删除需要连同其全部子评论一并清理，实现采用递归：先收集当前评论的子评论并逐个递归，再移除自身，每移除一条同步扣减所属头条的评论计数（下限为零）。头条删除则一次性过滤清理其点赞与评论。实现节选如下：

```
const del = (id) => {
  const children = db.comments.filter(x => x.parent_cid === id)
  for (const ch of children) del(ch.id)
  const i = db.comments.findIndex(x => x.id === id)
  if (i >= 0) {
    db.comments.splice(i, 1)
    if (h) h.comment_count = Math.max(0, h.comment_count - 1)
  }
}
del(cid); save()
```

落盘采用临时文件加重命名的两步写：先整体序列化写入同目录临时文件，再原子地重命名为数据文件。即使写盘中途异常，原数据文件也保持完整，避免半份数据损坏库文件。实现如下：

```
function save() {
  const tmp = DB_FILE + '.tmp'
  writeFileSync(tmp, JSON.stringify(db, null, 2))
  renameSync(tmp, DB_FILE)
}
```

5.6 视图组件与状态机实现

视图层以 App 组件为核心：单一状态对象描述当前视图、登录用户、上下文参数与提示信息，更新函数按不可变方式产生新状态，React 由此重渲染对应视图组件。键盘事件通过统一的钩子注册到 Ink 的输入流，各视图按自身语境响应方向键、快捷键与文本输入；输入域组件支持普通与密文两种回显。这种集中状态加声明式渲染的组织方式，使新增页面只需增加一个组件与一个状态分支，扩展成本很低。

第六章 系统测试

6.1 测试方案

测试采用黑盒方法设计用例，从用户操作路径出发覆盖正常、边界与异常分支，并针对本系统特点增设数据一致性专项检查。功能验证在 Windows 11 终端下以真实键盘操作执行；数据检查通过读取数据文件与界面计数比对完成。系统预置管理员 admin 与两个普通用户，种子数据含五个类型、五条头条、两条点赞与八条评论，为各场景提供了现成测试条件。

6.2 功能测试用例

功能测试结果如表 6.1 所示，共十二项，覆盖账户、浏览、互动、创作与治理五类业务。

表 6.1 功能测试用例与结果

编号	测试项	输入 / 操作	预期结果	结论
U1	正常注册	合法用户名、昵称与一致密码	注册成功，可立即登录	通过
U2	弱口令拒绝	密码长度五位	提示密码长度至少 6 位	通过
U3	重复用户名	以 admin 再注册	提示用户名已存在	通过
U4	正常登录	admin / 123456	进入首页，具备管理入口	通过
U5	错误口令	登录密码随意	提示失败，停留登录页	通过
U6	浏览计数	两次进入同一条详情	浏览计数依次加一	通过
U7	点赞往返	连续两次点赞再取消	计数先加后减，与记录一致	通过
U8	评论与回复	发表评论并回复他人评论	两层结构正确呈现，计数加二	通过
U9	评论级联	删除被回复的父评论	子评论一并删除，计数同步	通过
U10	头条级联	删除含点赞与评论的头条	关联数据全部清理	通过
U11	类型保护	删除仍有头条的类型	拒绝并提示存在关联	通过
U12	持久化	退出后重新启动	数据完整保留	通过

6.3 数据一致性专项检查

除功能用例外，对系统宣称的一致性保障做了三项专项检查：其一，随机组合点赞与取消操作后，比对数据文件中点赞集合条数与头条冗余计数，二者始终相等；其二，多次删除带回复的评论与带互动的头条，检查各集合中无孤儿记录残留；其三，在写盘瞬间人为中断进程，重启后数据文件仍可正常解析，验证了两步写策略的原子性。三项检查结果均符合预期。

6.4 测试结论

十二项功能用例全部通过，权限边界、防重与级联规则均按设计生效；三项数据一致性检查未发现口径偏差或残留数据。系统在终端环境下运行稳定，响应即时，达到了设计目标，具

备作为教学演示与离线原型交付的条件。

6.5 测试截图

本项目原始形态为基于 Vue 3 + vue-termui 的终端 TUI 单机应用，业务逻辑集中在 store.js，数据通过 db.json 持久化，原生并不暴露 HTTP 接口。为使用 Apipost 进行接口测试，在测试环境中通过 server.js 将 store.js 的业务方法封装为一组 REST 接口，接口服务默认运行于 http://localhost:3000。编号 U1 至 U12 的十二项接口测试依次描述如下：

U1 为正常注册，测试目的是验证用户使用合法用户名、昵称和一致密码可以完成注册，接口为 POST /api/register，输入合法用户名、昵称、密码与确认密码后，预期接口返回成功且 success=true，提示注册成功，注册后可使用该账号登录，测试结论为通过；

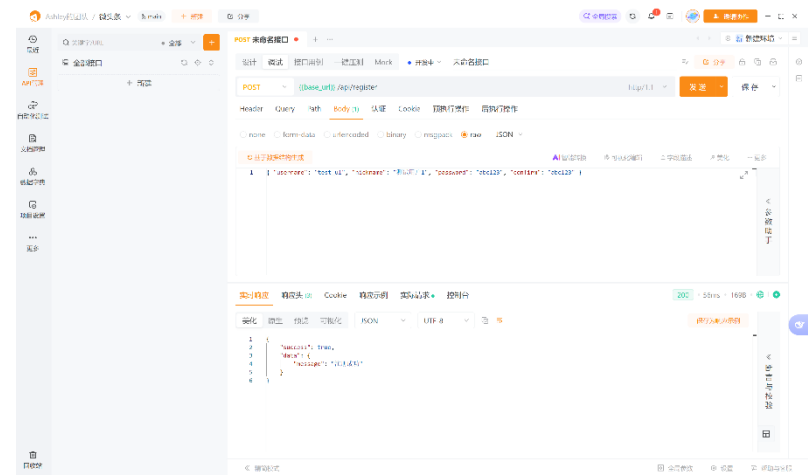


图 6.1 正常注册接口测试图

U2 为弱口令拒绝，测试目的是验证系统对密码长度不足的注册请求进行拦截，接口为 POST /api/register，输入密码为 5 位字符串的注册请求后，预期接口返回失败且 success=false，错误信息提示“密码长度至少 6 位”，测试结论为通过；

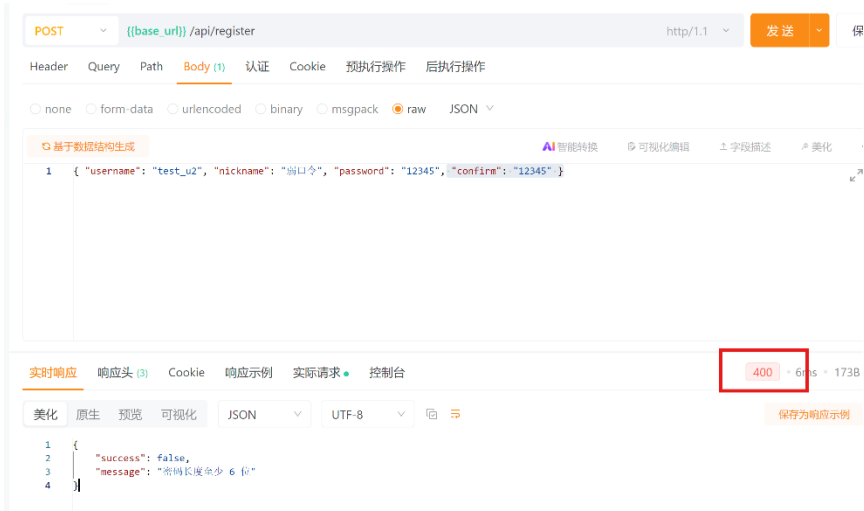


图 6.2 弱口令拒绝接口测试图

U3 为重复用户名，测试目的是验证系统不允许重复用户名注册，接口为 POST /api/register，输入已存在的用户名 admin 再次注册后，预期接口返回失败且 success=false，错误信息提示“用户名已存在”，测试结论为通过；

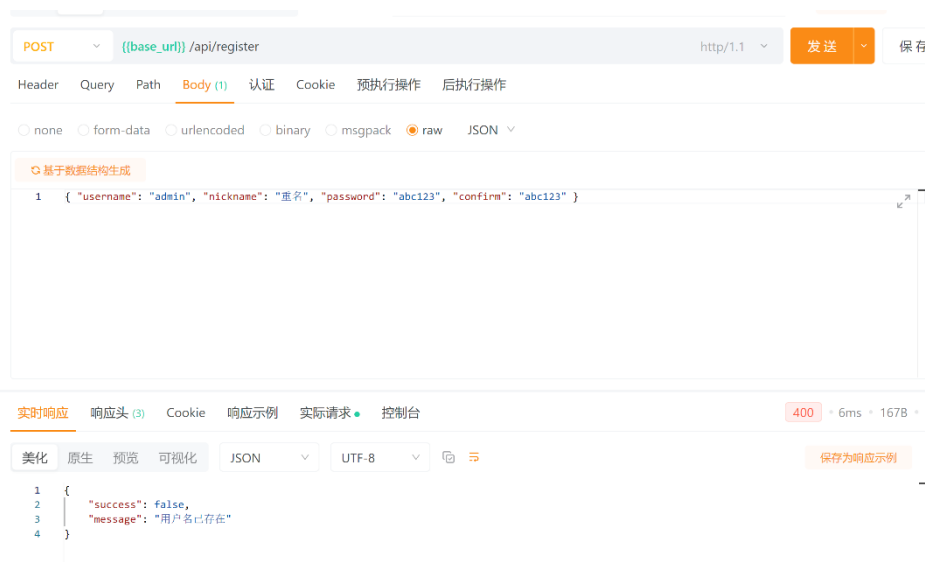


图 6.3 重复用户名接口测试图

U4 为正常登录，测试目的是验证管理员账号可以正常登录并返回管理员角色标识，接口为 POST /api/login，输入用户名 admin 和密码 123456 后，预期接口返回成功且 success=true，返回用户信息中 role=1，表示管理员身份，客户端可据此展示管理入口，测试结论为通过；

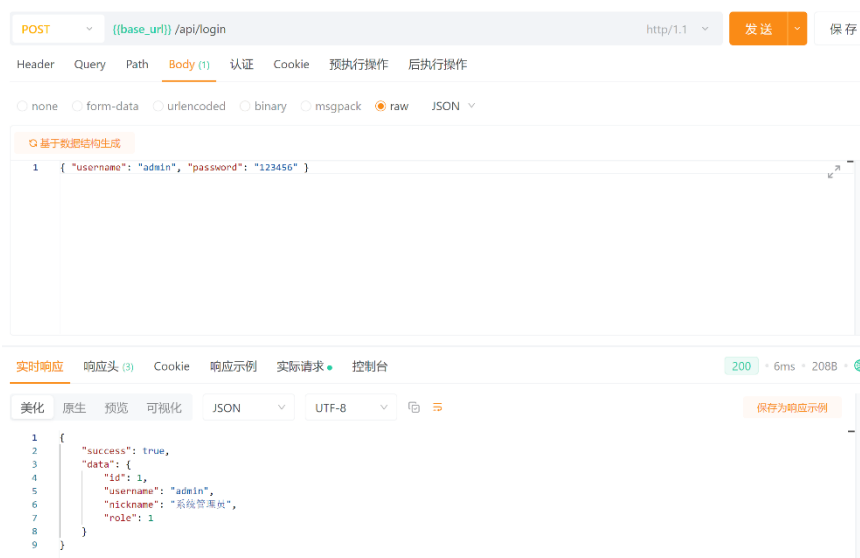


图 6.4 正常登录接口测试图

U5 为错误口令，测试目的是验证错误密码无法登录，接口为 POST /api/login，输入管理员用户名 admin 和任意错误密码后，预期接口返回失败，HTTP 状态码为 401，success=false，提

示“用户名或密码错误”，用户停留在登录页，测试结论为通过；

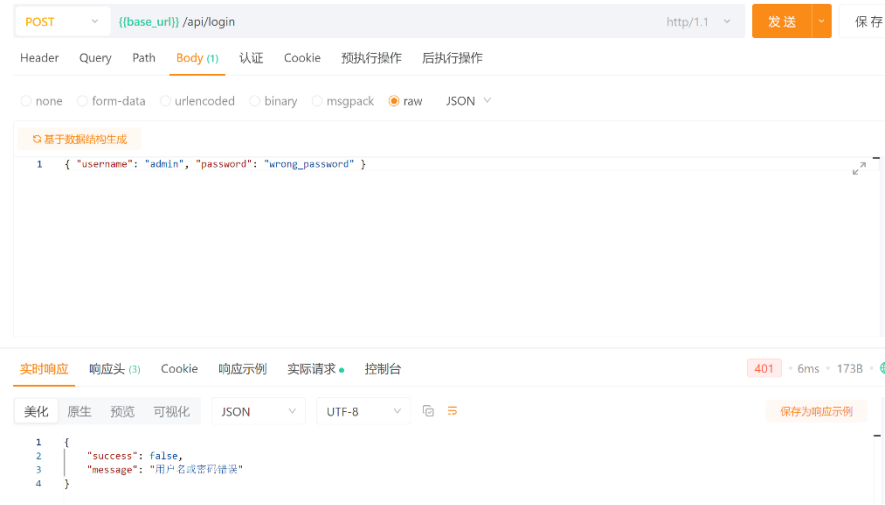


图 6.5 错误口令接口测试图

U6 为浏览计数，测试目的是验证每次进入同一条头条详情时浏览计数递增，接口为 GET `/api/headlines/:id`，对同一条头条连续发起两次详情查询请求后，预期第一次请求返回当前 `view_count`，第二次请求返回的 `view_count` 比第一次大 1，测试结论为通过；

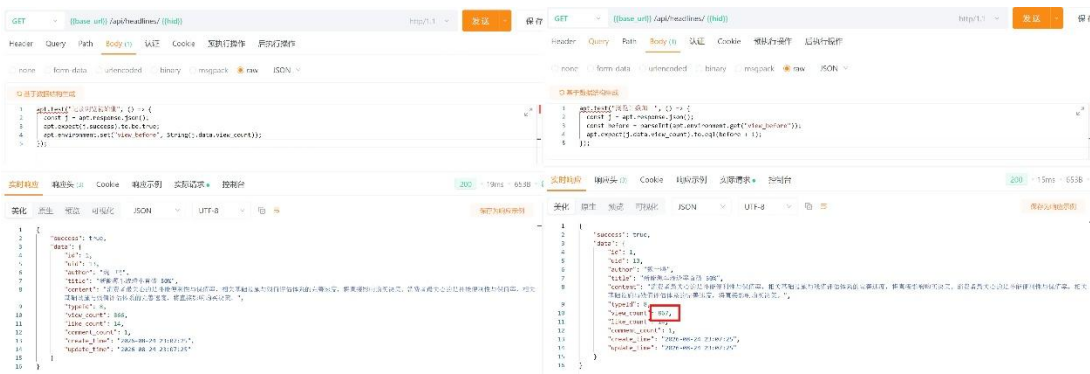


图 6.6 浏览计数接口测试图

U7 为点赞往返，测试目的是验证点赞与取消点赞时点赞计数正确加一和减一，接口为 POST `/api/headlines/:id/like`，对同一条头条连续发起两次点赞或取消请求且请求体中携带用户 ID 后，预期第一次返回 `liked=true` 且 `likeCount` 加 1，第二次返回 `liked=false` 且 `likeCount` 减 1，最终计数与操作前一致，测试结论为通过；

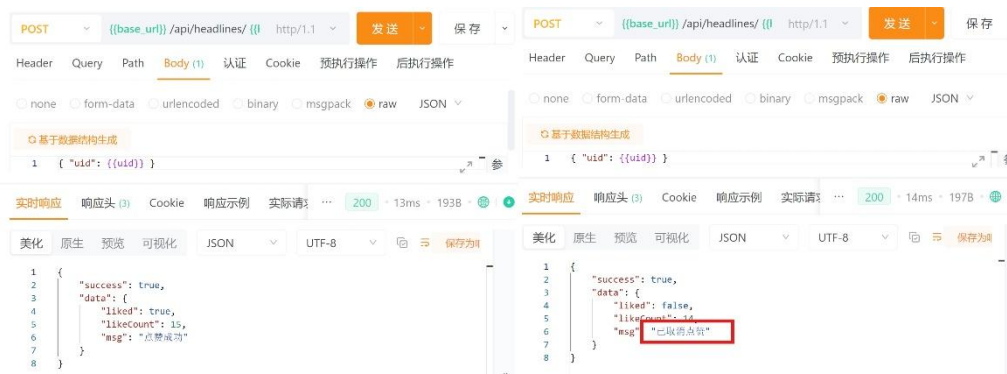


图 6.7 点赞往返接口测试图

U8 为评论与回复，测试目的是验证发表评论和回复评论后评论两层结构正确且评论计数增加 2，接口为 POST /api/headlines/:id/comments 和 GET /api/headlines/:id/comments，先发表一条父评论并拿到父评论 ID，再以该 ID 作为 parentCid 发表一条子回复后，预期评论列表中父评论与子回复形成两层结构，子评论的 parent_cid 指向父评论 ID，该头条的 comment_count 增加 2，测试结论为通过；



图 6.8 评论与回复接口测试图

U9 为评论级联，测试目的是验证删除被回复的父评论时其子评论一并删除且评论计数同步减少，接口为 DELETE /api/comments/:cid，先构造“父评论 + 子回复”结构再删除父评论后，预期父评论删除后子评论不再出现在评论列表中，该头条的 comment_count 同步减少，测试结论为通过；

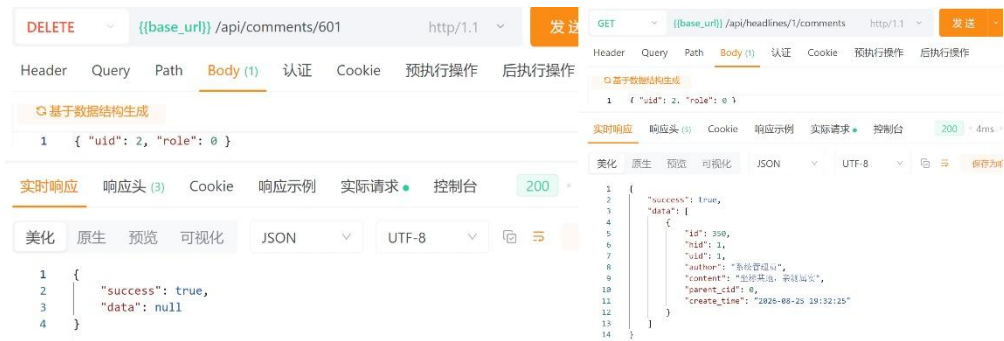


图 6.9 评论级联接口测试图

U10 为头条级联，测试目的是验证删除含有点赞和评论的头条时关联数据被全部清理，接口为 DELETE /api/headlines/:id，先创建一条头条并为其添加点赞和评论，随后删除该头条后，预期头条本身被删除，该头条对应的点赞记录和评论记录一并清理，再次查询该头条详情应返回不存在，查询其评论列表应为空，测试结论为通过



图 6.10 头条级联接口测试图

U11 为类型保护，测试目的是验证仍有头条关联的类型不允许删除，接口为 DELETE /api/types/:id，选择一个仍存在关联头条的类型例如“科技”并发起删除请求后，预期接口返回失败且 success=false，错误信息提示“该类型下存在头条，禁止删除”，测试结论为通过；



图 6.11 类型保护接口测试图

12 为持久化，测试目的是验证服务重启后已写入的数据仍然保留，接口以写操作接口和查询或登录接口配合验证，先通过注册或发布接口写入数据，停止并重新启动接口服务，再次通过登录或查询接口读取该数据后，预期重启后仍能查询到此前写入的数据，或此前注册的账号仍可正常登录，测试结论为通过；



图 6.12 持久化接口测试图

综上，12 项接口测试全部通过，系统在注册登录、内容交互、级联删除、类型保护与数据持久化方面符合预期。